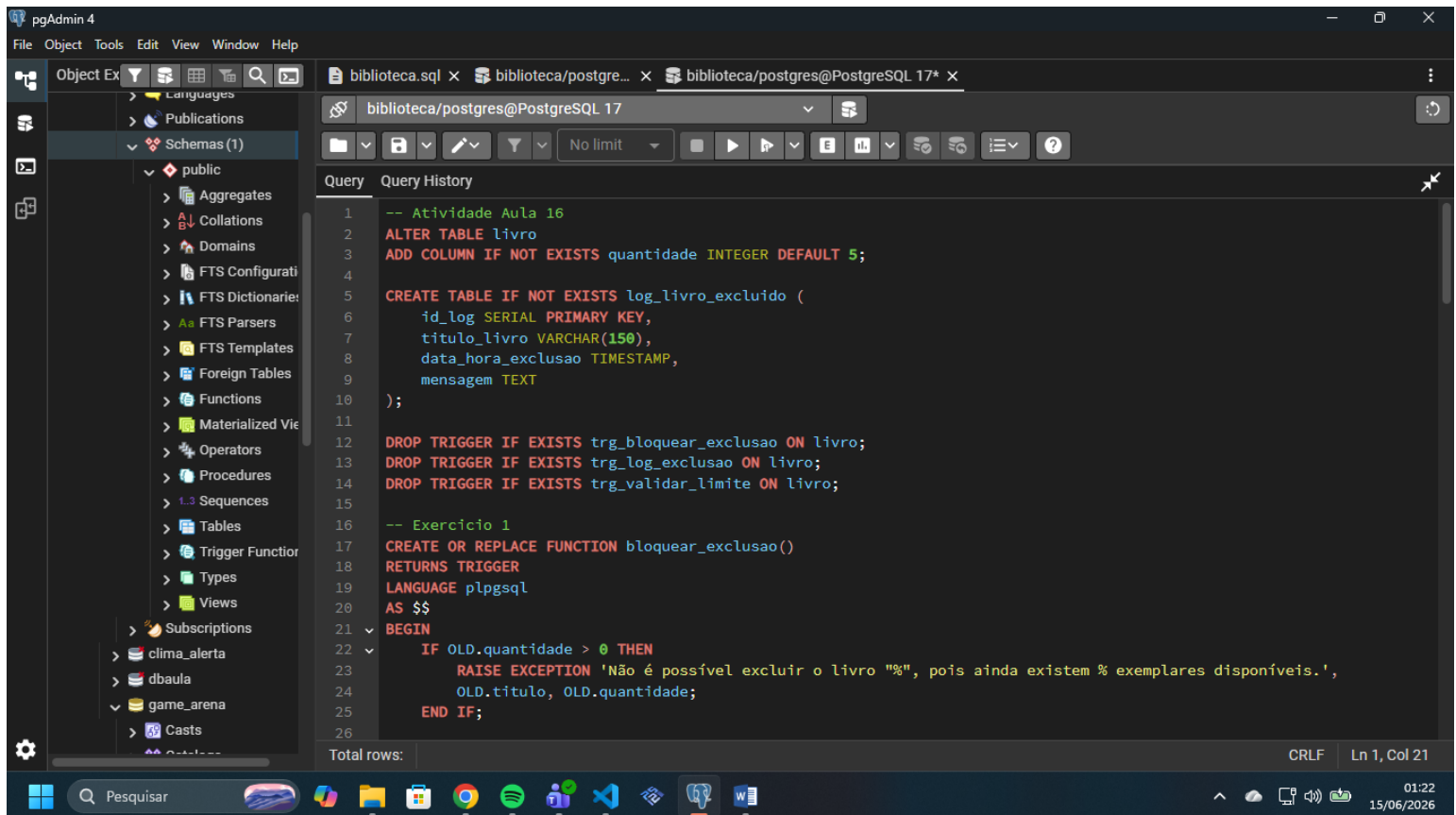


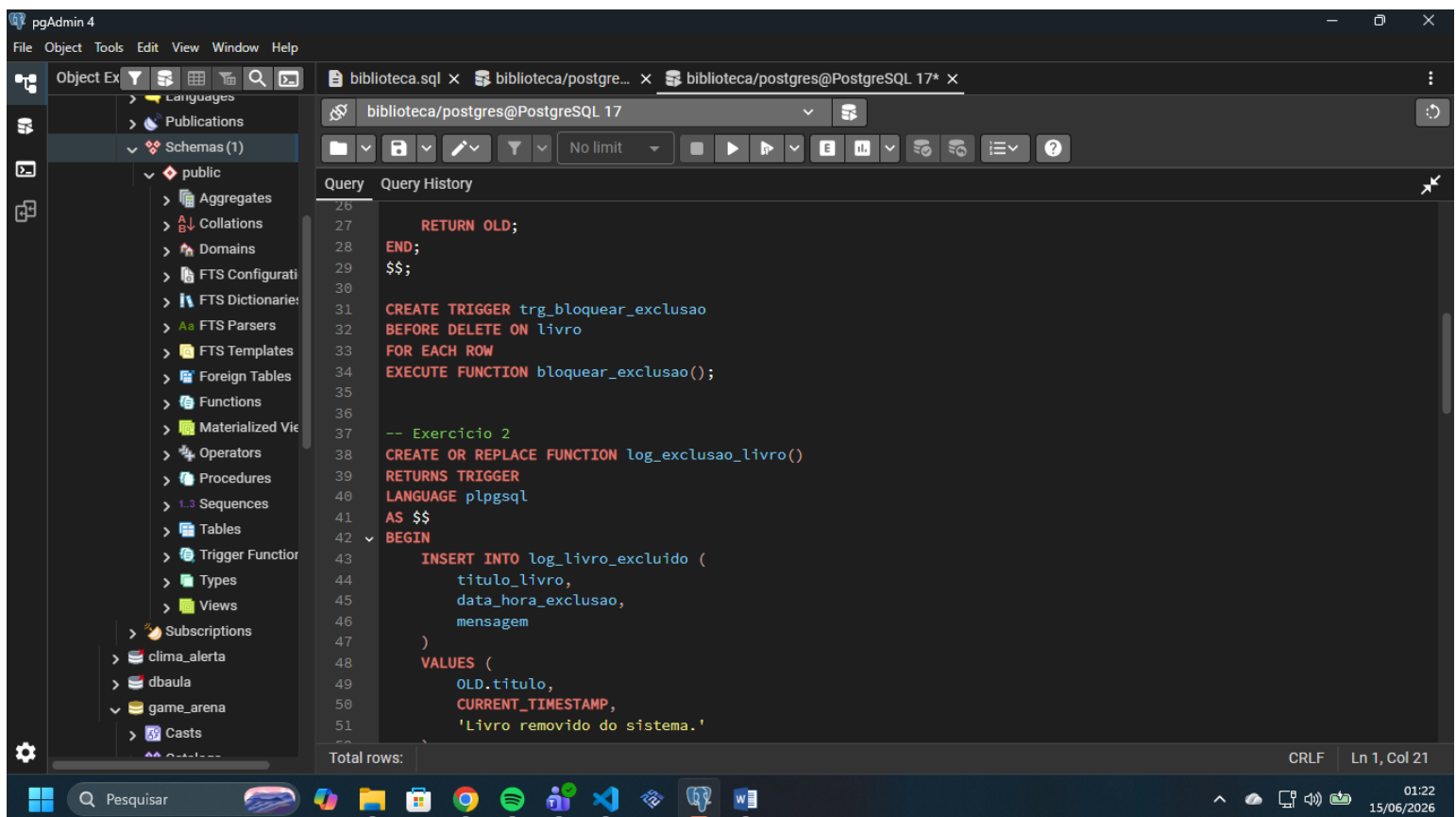
BDR-ATIVIDADE-AULA16-ISRAEL-LEMES



The screenshot shows the pgAdmin 4 interface with the following SQL queries in the query editor:

```
1  -- Atividade Aula 16
2  ALTER TABLE livro
3  ADD COLUMN IF NOT EXISTS quantidade INTEGER DEFAULT 5;
4
5  CREATE TABLE IF NOT EXISTS log_livro_excluido (
6      id_log SERIAL PRIMARY KEY,
7      titulo_livro VARCHAR(150),
8      data_hora_exclusao TIMESTAMP,
9      mensagem TEXT
10 );
11
12 DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;
13 DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;
14 DROP TRIGGER IF EXISTS trg_validar_limite ON livro;
15
16 -- Exercício 1
17 CREATE OR REPLACE FUNCTION bloquear_exclusao()
18 RETURNS TRIGGER
19 LANGUAGE plpgsql
20 AS $$
21 BEGIN
22     IF OLD.quantidade > 0 THEN
23         RAISE EXCEPTION 'Não é possível excluir o livro "%", pois ainda existem % exemplares disponíveis.',
24             OLD.titulo, OLD.quantidade;
25     END IF;
26
```

The interface includes a sidebar with a tree view of database objects (Schemas, public, Aggregates, Collations, Domains, FTS Configurati, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables, Trigger Functions, Types, Views, Subscriptions, clima_alerta, dbaula, game_arena, Casts) and a bottom status bar showing 'Total rows: CRLF Ln 1, Col 21'.



The screenshot shows the pgAdmin 4 interface with the following SQL queries in the query editor:

```
26
27     RETURN OLD;
28 END;
29 $$;
30
31 CREATE TRIGGER trg_bloquear_exclusao
32 BEFORE DELETE ON livro
33 FOR EACH ROW
34 EXECUTE FUNCTION bloquear_exclusao();
35
36 -- Exercício 2
37 CREATE OR REPLACE FUNCTION log_exclusao_livro()
38 RETURNS TRIGGER
39 LANGUAGE plpgsql
40 AS $$
41 BEGIN
42     INSERT INTO log_livro_excluido (
43         titulo_livro,
44         data_hora_exclusao,
45         mensagem
46     )
47     VALUES (
48         OLD.titulo,
49         CURRENT_TIMESTAMP,
50         'Livro removido do sistema.'
51     );
52
```

The interface includes a sidebar with a tree view of database objects (Schemas, public, Aggregates, Collations, Domains, FTS Configurati, FTS Dictionaries, FTS Parsers, FTS Templates, Foreign Tables, Functions, Materialized Views, Operators, Procedures, Sequences, Tables, Trigger Functions, Types, Views, Subscriptions, clima_alerta, dbaula, game_arena, Casts) and a bottom status bar showing 'Total rows: CRLF Ln 1, Col 21'.

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurati
 - FTS Dictionarie
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Vie
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Function
 - Types
 - Views
 - Subscriptions
 - clima_alerta
 - dbaula
 - game_arena
 - Casts

Query

```
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
```

```
RETURN OLD;
END;
$$;

CREATE TRIGGER trg_log_exclusao
AFTER DELETE ON livro
FOR EACH ROW
EXECUTE FUNCTION log_exclusao_livro();

-- Exercício 3
CREATE OR REPLACE FUNCTION validar_limite_estoque()
RETURNS TRIGGER
LANGUAGE plpgsql
AS $$
BEGIN
    IF NEW.quantidade > 100 THEN
        RAISE EXCEPTION 'Quantidade inválida. O estoque não pode ser maior que 100.';
    END IF;

    RETURN NEW;
END;
$$;

CREATE TRIGGER trg_validar_limite
```

Total rows: CRLF Ln 65, Col 48

Pesquisar

01:22 15/06/2026

pgAdmin 4

File Object Tools Edit View Window Help

Object Explorer

- Publications
- Schemas (1)
 - public
 - Aggregates
 - Collations
 - Domains
 - FTS Configurati
 - FTS Dictionarie
 - FTS Parsers
 - FTS Templates
 - Foreign Tables
 - Functions
 - Materialized Vie
 - Operators
 - Procedures
 - Sequences
 - Tables
 - Trigger Function
 - Types
 - Views
 - Subscriptions
 - clima_alerta
 - dbaula
 - game_arena
 - Casts

Query

```
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
```

```
BEFORE UPDATE ON livro
FOR EACH ROW
EXECUTE FUNCTION validar_limite_estoque();

-- Testes

-- Teste do exercício 1
DELETE FROM livro
WHERE id_livro = 1;

-- Teste do exercício 3
UPDATE livro
SET quantidade = 150
WHERE id_livro = 1;

-- Teste do exercício 2: para funcionar, precisa zerar a quantidade antes
UPDATE livro
SET quantidade = 0
WHERE id_livro = 4;

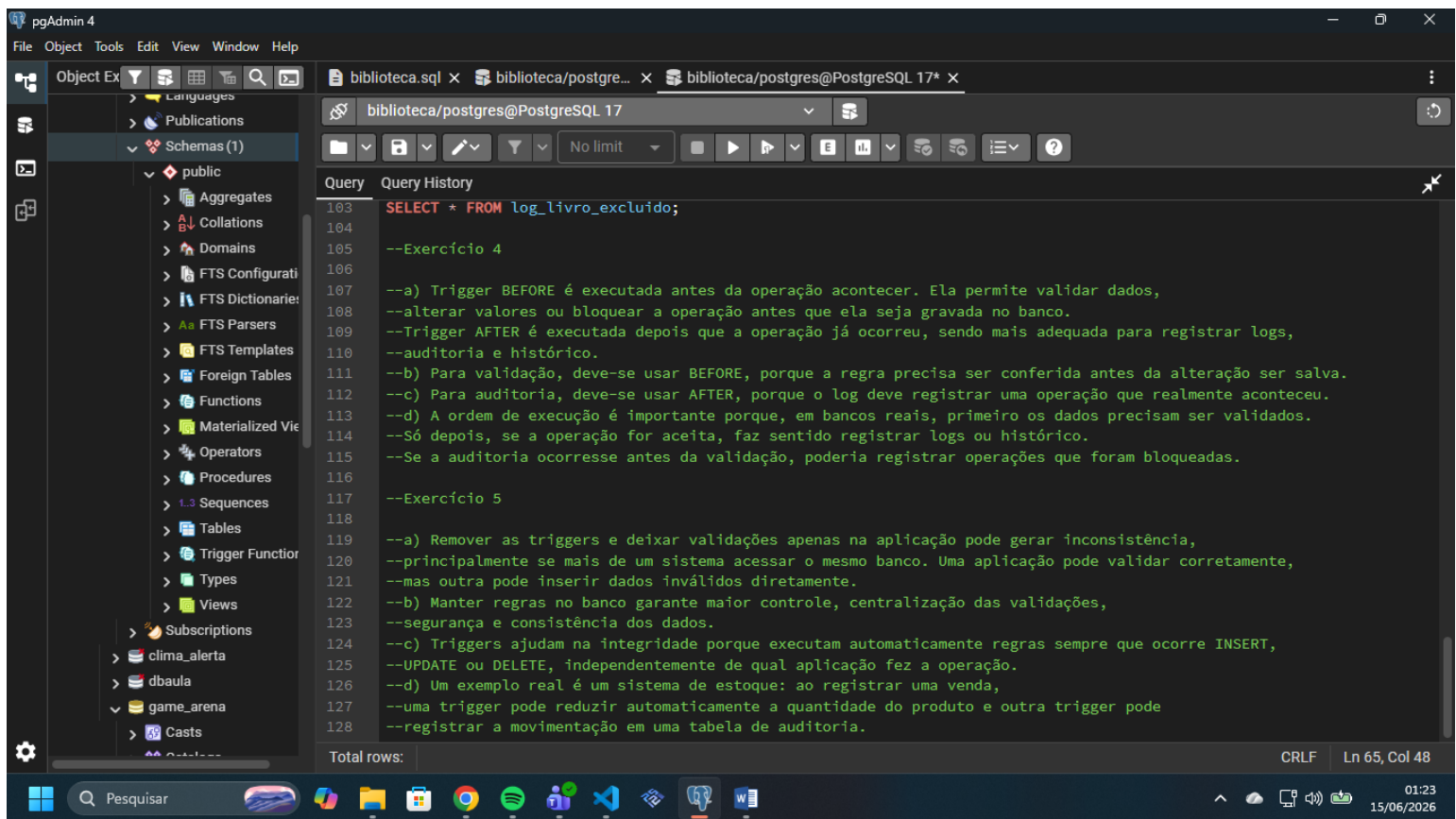
DELETE FROM livro
WHERE id_livro = 4;

SELECT * FROM log_livro_excluido;
```

Total rows: CRLF Ln 65, Col 48

Pesquisar

01:22 15/06/2026



-- Atividade Aula 16

ALTER TABLE livro

ADD COLUMN IF NOT EXISTS quantidade INTEGER DEFAULT 5;

CREATE TABLE IF NOT EXISTS log_livro_excluido (

id_log SERIAL PRIMARY KEY,

titulo_livro VARCHAR(150),

data_hora_exclusao TIMESTAMP,

mensagem TEXT

);

DROP TRIGGER IF EXISTS trg_bloquear_exclusao ON livro;

DROP TRIGGER IF EXISTS trg_log_exclusao ON livro;

DROP TRIGGER IF EXISTS trg_validar_limite ON livro;

-- Exercício 1

CREATE OR REPLACE FUNCTION bloquear_exclusao()

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

BEGIN

IF OLD.quantidade > 0 THEN

RAISE EXCEPTION 'Não é possível excluir o livro "%", pois ainda existem % exemplares disponíveis.',

OLD.titulo, OLD.quantidade;

END IF;

RETURN OLD;

END;

\$\$;

CREATE TRIGGER trg_bloquear_exclusao

BEFORE DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION bloquear_exclusao();

-- Exercício 2

CREATE OR REPLACE FUNCTION log_exclusao_livro()

RETURNS TRIGGER

```
LANGUAGE plpgsql

AS $$

BEGIN

    INSERT INTO log_livro_excluido (

        titulo_livro,

        data_hora_exclusao,

        mensagem

    )

VALUES (

    OLD.titulo,

    CURRENT_TIMESTAMP,

    'Livro removido do sistema.'

);

RETURN OLD;

END;

$$;


CREATE TRIGGER trg_log_exclusao

AFTER DELETE ON livro

FOR EACH ROW

EXECUTE FUNCTION log_exclusao_livro();
```

-- Exercício 3

```
CREATE OR REPLACE FUNCTION validar_limite_estoque()
```

RETURNS TRIGGER

LANGUAGE plpgsql

AS \$\$

BEGIN

IF NEW.quantidade > 100 THEN

RAISE EXCEPTION 'Quantidade inválida. O estoque não pode ser maior que 100.';

END IF;

RETURN NEW;

END;

\$\$;

CREATE TRIGGER trg_validar_limite

BEFORE UPDATE ON livro

FOR EACH ROW

EXECUTE FUNCTION validar_limite_estoque();

-- Testes

-- Teste do exercicio 1

DELETE FROM livro

WHERE id_livro = 1;

-- Teste do exercicio 3

UPDATE livro

SET quantidade = 150

WHERE id_livro = 1;

-- Teste do exercicio 2: para funcionar, precisa zerar a quantidade antes

UPDATE livro

SET quantidade = 0

WHERE id_livro = 4;

DELETE FROM livro

WHERE id_livro = 4;

SELECT * FROM log_livro_excluido;

--Exercício 4

--a) Trigger BEFORE é executada antes da operação acontecer. Ela permite validar dados,

--alterar valores ou bloquear a operação antes que ela seja gravada no banco.

--Trigger AFTER é executada depois que a operação já ocorreu, sendo mais adequada para registrar logs,

--auditoria e histórico.

--b) Para validação, deve-se usar BEFORE, porque a regra precisa ser conferida antes da alteração ser salva.

--c) Para auditoria, deve-se usar AFTER, porque o log deve registrar uma operação que realmente aconteceu.

--d) A ordem de execução é importante porque, em bancos reais, primeiro os dados precisam ser validados.

--Só depois, se a operação for aceita, faz sentido registrar logs ou histórico.

--Se a auditoria ocorresse antes da validação, poderia registrar operações que foram bloqueadas.

--Exercício 5

- a) Remover as triggers e deixar validações apenas na aplicação pode gerar inconsistência,
--principalmente se mais de um sistema acessar o mesmo banco. Uma aplicação pode validar corretamente,
--mas outra pode inserir dados inválidos diretamente.
- b) Manter regras no banco garante maior controle, centralização das validações,
--segurança e consistência dos dados.
- c) Triggers ajudam na integridade porque executam automaticamente regras sempre que ocorre INSERT,
--UPDATE ou DELETE, independentemente de qual aplicação fez a operação.
- d) Um exemplo real é um sistema de estoque: ao registrar uma venda,
--uma trigger pode reduzir automaticamente a quantidade do produto e outra trigger pode
--registrar a movimentação em uma tabela de auditoria._completo('Livro Inválido', 2025, 1, 1, -50);